

HACK THE BOX SHERLOCK - DFIR

OPERATION BLACKOUT 2025: GHOST THREAD

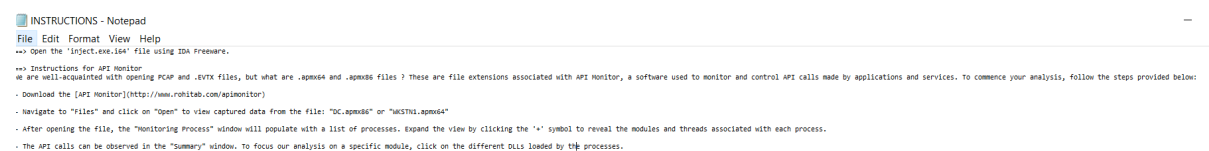
Sherlock Scenario

Byte Doctor suspects the attacker used a process injection technique to run malicious code within a legitimate process, leaving minimal traces on the file system. The logs reveal Win32 API calls that hint at a specific injection method used in the attack.

Your task is to analyze these logs using a tool called API Monitor to uncover the injection technique and identify which legitimate process was targeted.

s PC > Desktop > HacktheBox Sherlock > GhostThread				Search GhostThread
Name	Date modified	Type	Size	
 Ghost-Thread.apmx64	4/17/2025 12:41 AM	APMX64 File	3,483 KB	
 inject.exe.i64	4/16/2025 11:29 PM	I64 File	2,275 KB	
 INSTRUCTIONS	4/17/2025 12:52 AM	Text Document	1 KB	

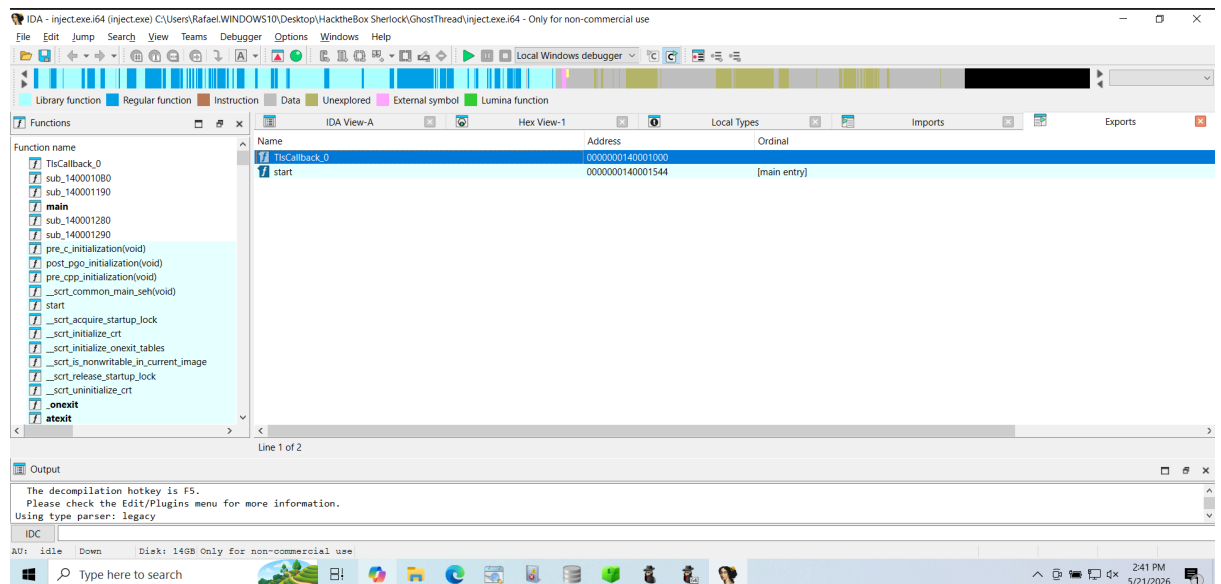
After unzipping the zip file I got from HackTheBox, I seem to get 3 files with an INSTRUCTIONS.txt file and I opened it and it showed me how to do this Sherlock which is to use API Monitor just like described in the Sherlock Scenario.



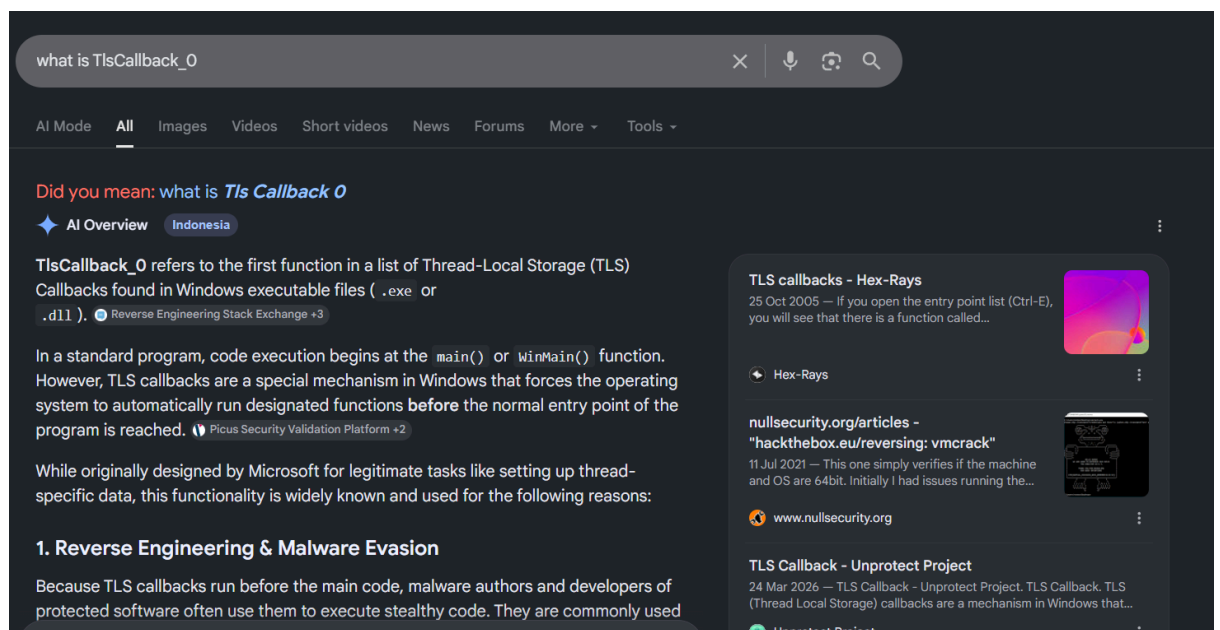
The setup for the files of this Sherlock is more complicated than usual. I had to make an account at <https://hex-rays.com/> and install IDA Free and for the Individual, non-commercial use version, I had to request an IDA Free License Key. Then I downloaded the IDA Freeware and the hexlic file used to activate it.

Then we will use API Monitor and IDA Freeware to answer the 7 questions below. And seems to be this Sherlock will have to do with reverse engineering.

1. What process injection technique did the attacker use?

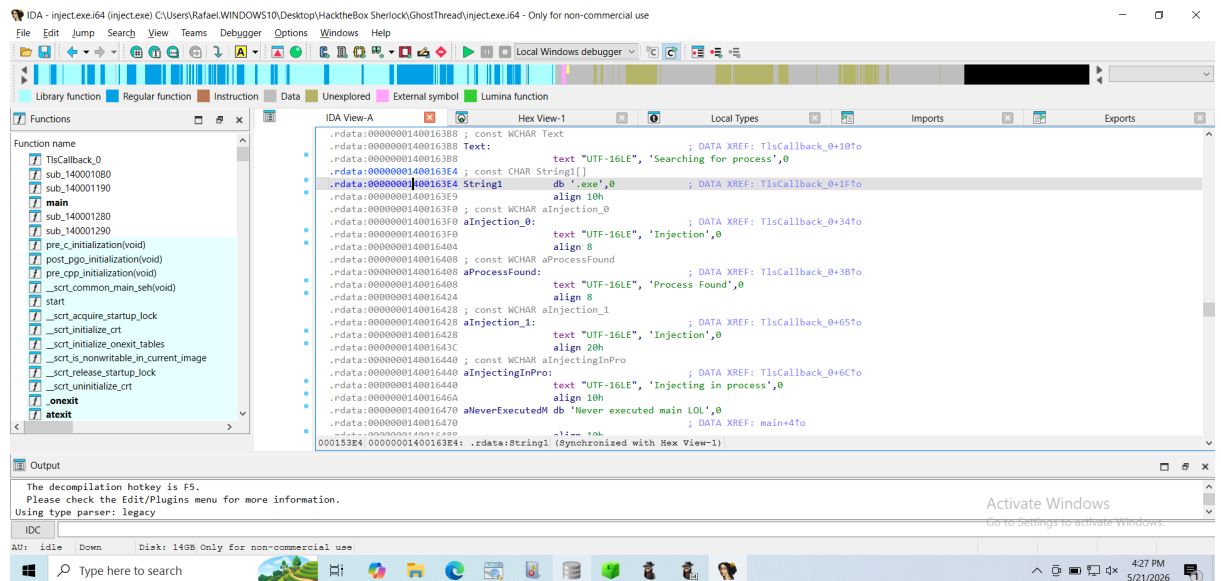


I started off with the inject.exe.i4 file by opening it on IDA Freeware and then I noticed that there is an Export named TlsCallback_0, which then I Googled what that is.



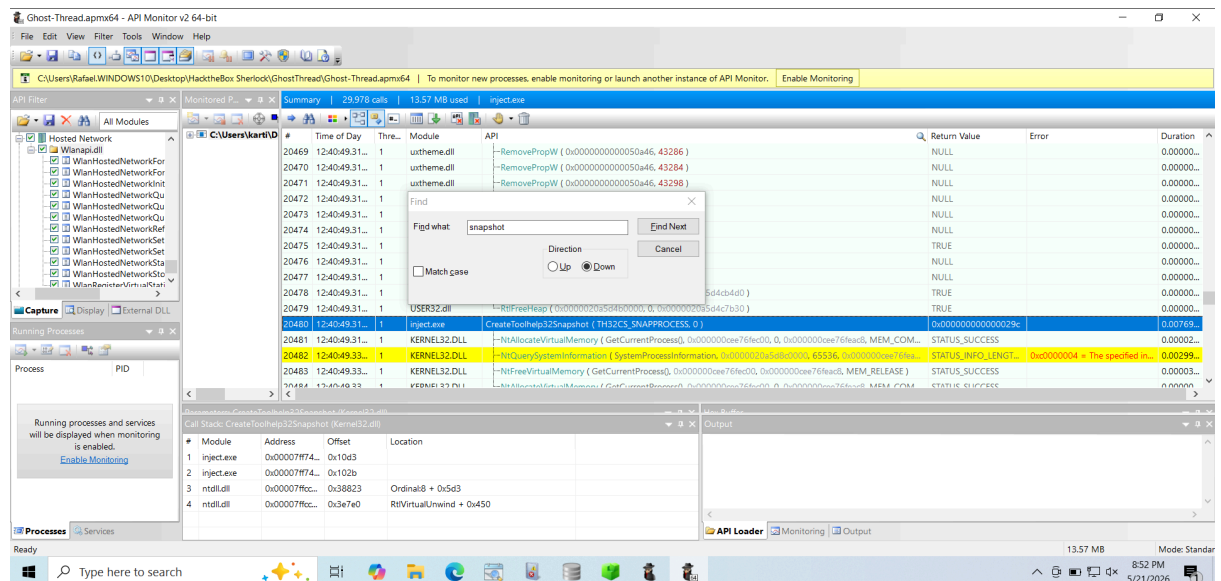
It stands for Thread Local Storage Callback, in which normally a program would begin from `main()` or `WinMain()` function but then the TLS callbacks execute the chosen functions automatically before the normal entry point of other programs.

Rafael Angelo Christianto



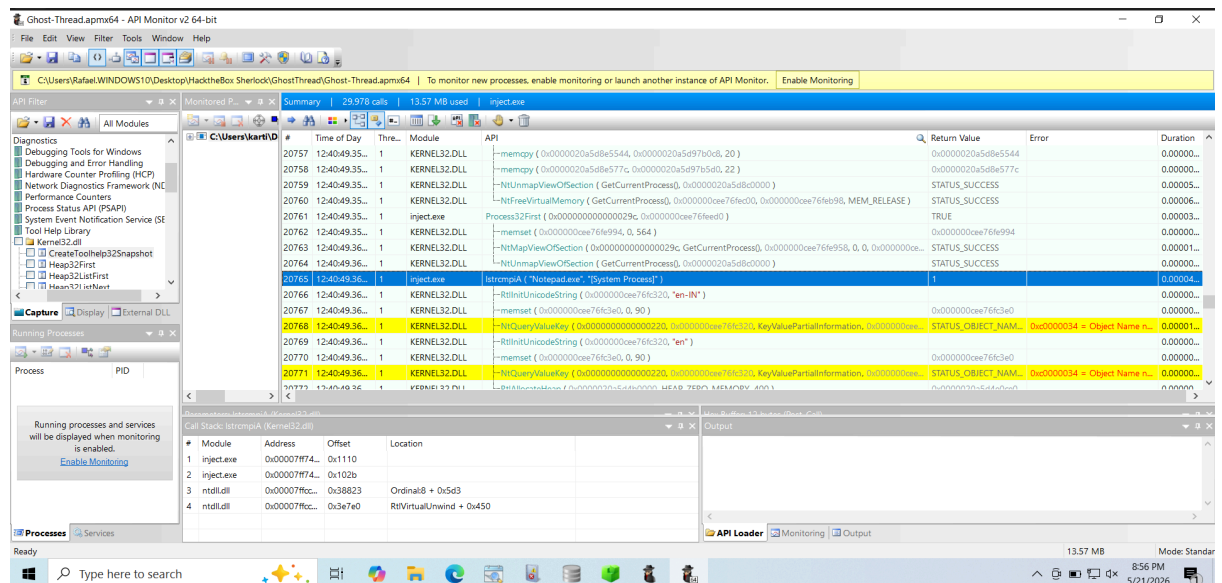
After following it, we see the Injection process after the process is found. Therefore the answer is **Thread Local Storage**

2. Which Win32 API was used to take snapshots of all processes and threads on the system?

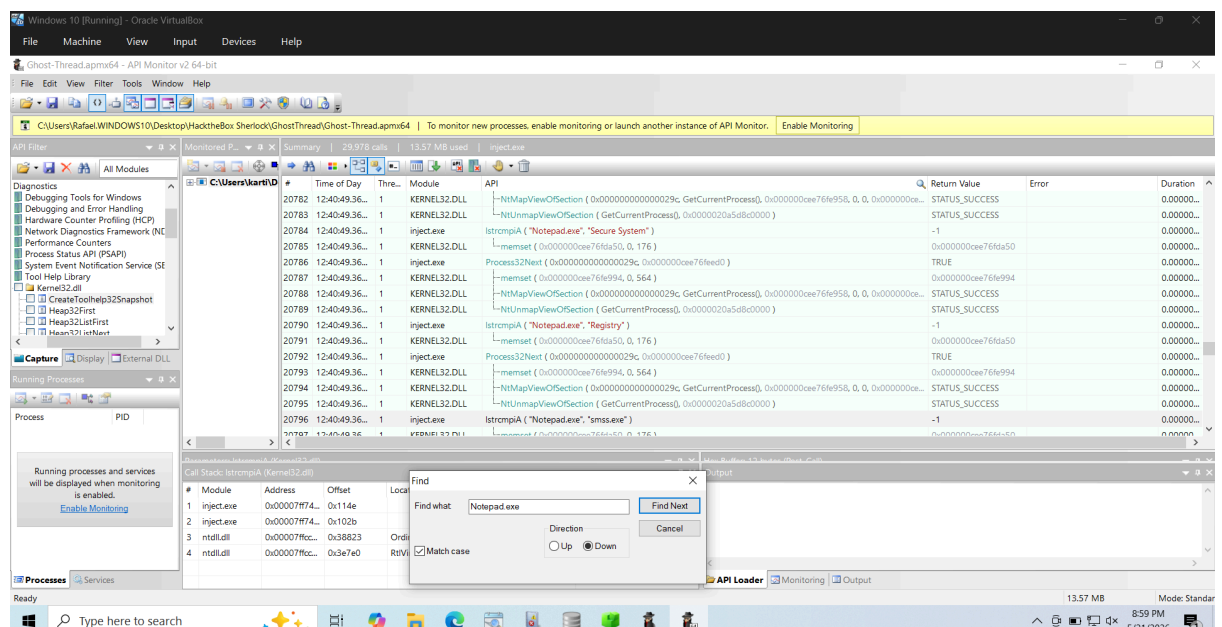


Since this question is related to API, I transitioned to the API Monitor for the file given by the Sherlock named "Ghost-Thread" and then I searched up for the keyword "snapshot" and then I found the answer which is **CreateToolhelp32Snapshot**, which is a Windows API used to capture snapshot for its current state including its processes, threads, heaps and modules

3. Which process is the attacker's binary attempting to locate for payload injection?



Still under the Ghost-Thread API Monitor file, I scrolled down more and then I saw a module named inject.exe found a process called Notepad.exe getting compared by using IstrcmpA and there is an enumeration of all the processes by Process32Next



In the screenshot above, we can see that the attacker is trying to enumerate the processes running and comparing it whether they are a match with Notepad.exe which would be good for payload injection because every system has one.

22374	12:40:49.49...	1	inject.exe	IstrcmpiA ("Notepad.exe", "Notepad.exe")	0	0.00000...
22375	12:40:49.49...	1	KERNEL32.DLL	memset (0x00000000, 0, 176)	0x00000000	0.00000...
22376	12:40:49.49...	1	inject.exe	CloseHandle (0x000000000000029c)	TRUE	0.00005...
22377	12:40:49.49...	1	KERNELBASE.dll	NtClose (0x000000000000029c)	STATUS_SUCCESS	0.00004...
22378	12:40:49.49...	1	inject.exe	MessageBoxW (NULL, "Process Found", "Injection", MB_OK)	IDOK	0.99732...

And that is why when they found Notepad.exe it says Process Found.

4. What is the process ID of the identified process?

The screenshot shows the API Monitor interface with the API call IstrcmpiA selected. The Parameters tab is expanded, showing the Post-Call Value as 16224. This value is the ProcessID of the target process, Notepad.exe.

This one is more tricky. The answer is not found on the IstrcmpiA where it found the Notepad.exe match. However it is found on the Process32Next where it can be found on the Parameters specifically on the Post-Call Value we see the ProcessID which is **16224**

5. What is the size of the shellcode?

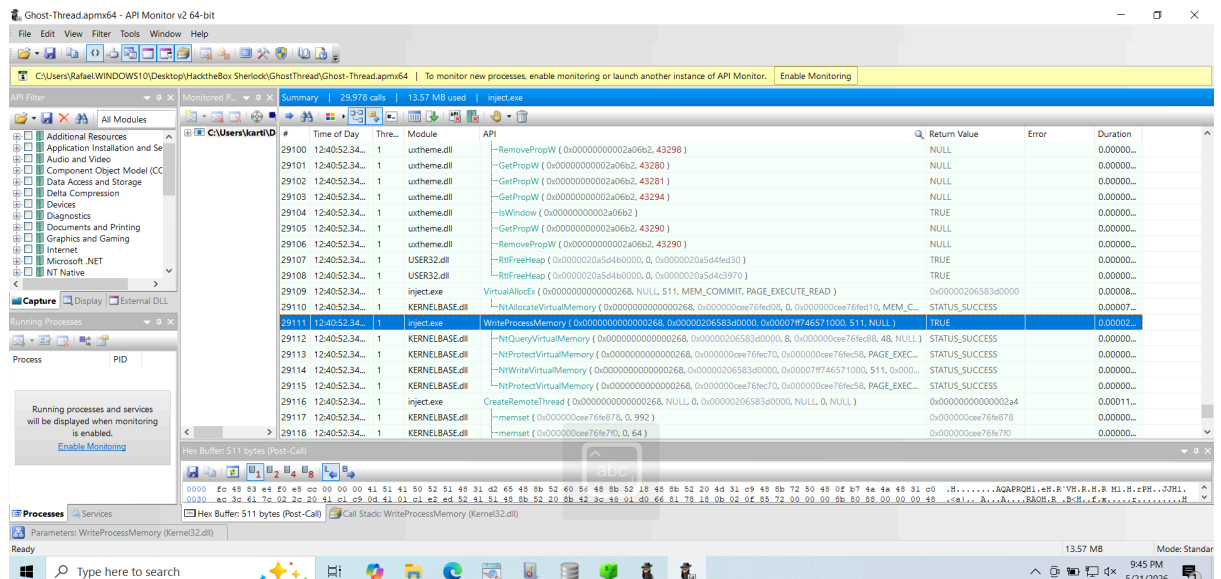
3. Payload Delivery (Writing)

Once the memory is allocated, the injector physically copies the shellcode into that remote memory space. Palo Alto Networks +1

- WriteProcessMemory** : Writes the malicious code to the target.
- NtWriteVirtualMemory** (Low-level NT equivalent). Palo Alto Networks +2

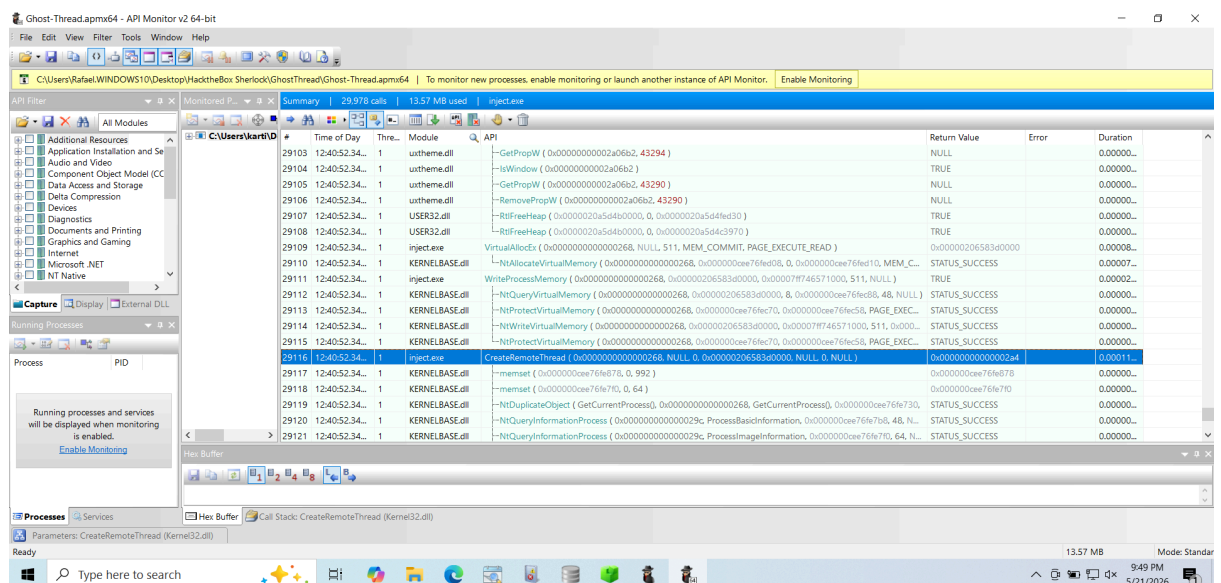
I Googled injecting malicious code or shellcode I will search WriteProcessMemory in the API Monitor

Rafael Angelo Christianto

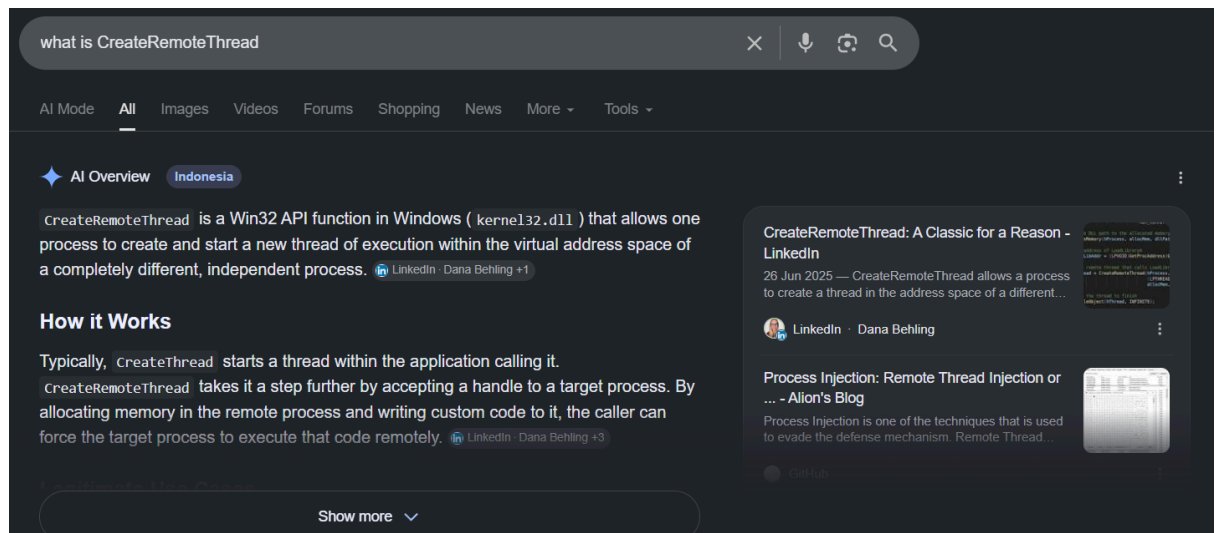


And then searching with the direction going down from the Notepad.exe enumeration row, I find this row for WriteProcessMemory where it contains a buffer argument and that buffer will contain the shellcode, and the size can be seen in the row, or bottom left beside the Hex Buffer text which is **511** bytes

6. Which Win32 API was used to execute the injected payload in the identified process?

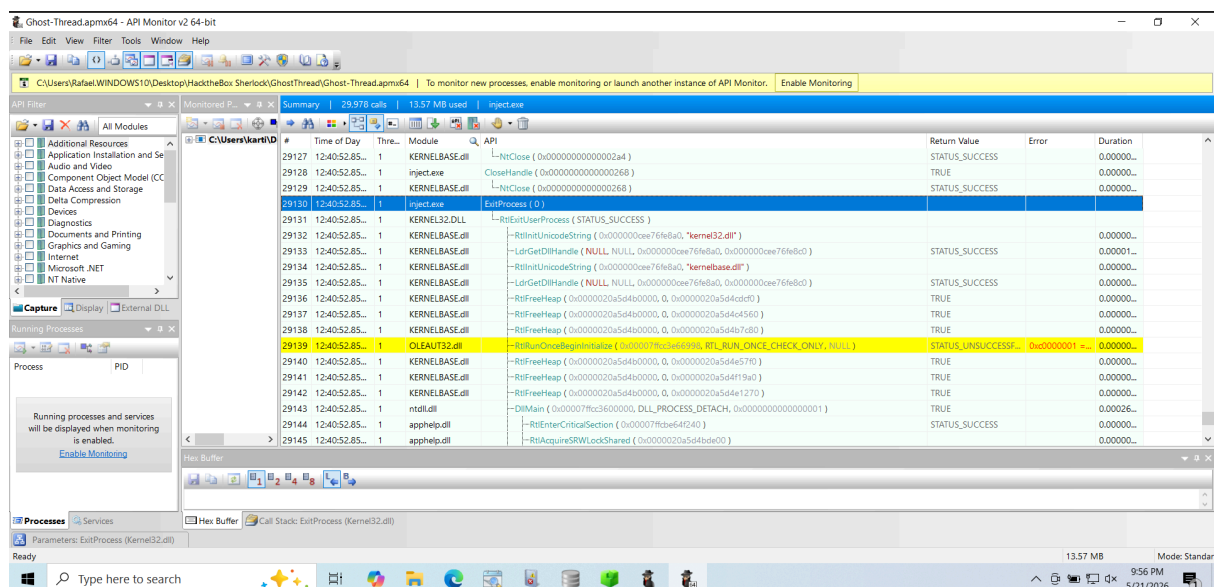


Not far below after the payload delivery from the previous question I see the module inject.exe doing another Win32 API which is CreateRemoteThread and I Googled what it is.



CreateRemoteThread is a Win32 API function in Windows that allows a process to start a new thread of executing a different process, so it suits this question and therefore the answer is **CreateRemoteThread**

- The injection method used by the attacker executes before the main() function is called. Which Win32 API is responsible for terminating the program before main() runs?



Scrolling more downwards, I find the Win32 API of ExitProcess under CloseHandle which both names seems to be possible.

ExitProcess win32 api

AI Mode **All** Videos Images Shopping Forums Short videos More Tools

 Microsoft Learn
https://learn.microsoft.com > ... > Processthreadsapi.h

ExitProcess function (processthreadsapi.h) - Win32 apps

31 Oct 2022 — Use the GetExitCodeProcess function to retrieve the process's exit value. Use the GetExitCodeThread function to retrieve a thread's exit value. [Read more](#)

AI Overview **Indonesia**  +7

The `ExitProcess` function is the standard Win32 API used to terminate the calling process and all its associated threads cleanly. It is defined in the `processthreadsapi.h` header and exported by `kernel32.dll`.  Microsoft Learn +2

After Googling it, it says that `ExitProcess` is a function that terminates the calling process and its associated threads which in this case exits the process of the `inject.exe` and then therefore the answer to this question is **ExitProcess**



You have solved Operation Blackout 2025: Ghost Thread!

Congratulations  **rafaelangelo** best of luck in capturing flags ahead!

#511	21 May 2026	Retired
Sherlock Rank	Solve Date	Sherlock State

Ok Share